

Databricks under the hood and legacy Big Data Systems

What goes on behind the scenes of Databricks that you do not see.

Ever wonder how a cluster is spun up without starting, configuring or stopping it? Why Kubernetes is so important? How Map Reduce really works?

Lets begin with starting a Spark Session aka kicking off the Databricks Cluster.

When you push “start cluster” on your Databricks Compute tab you are creating a spark session as shown below. Notice the configurations. I am sure everyone has added configuration to their delta table to add enable logging for historical checkpoints.

Databricks version of adding log retention...I am sure you have done this

```
spark.sql("ALTER TABLE delta.`/path/to/table` SET TBLPROPERTIES  
('delta.logRetentionDuration' = 'interval 90 days')")
```

The above configuration would be placed in the builder below if you were really running a legacy system or wanted more control.

Now, let's move onto a pipeline example. In this example I am pulling from Yahoo Finance API, performing some transformations and saving it to a local Hive Metastore. Now, we have Unity Catalog which has replaced traditional Hive Metastore.

```
import yfinance as yf
import numpy as np
import pandas as pd
import datetime
from spark_session import get_spark
from pyspark.sql import functions as F
from delta.tables import DeltaTable

TICKERS = {
    'GC=F': 'Gold',
    'SI=F': 'Silver',
    'PL=F': 'Platinum',
    'PA=F': 'Palladium',
    'HG=F': 'Copper',
    'CL=F': 'WTI Crude Oil',
    'BZ=F': 'Brent Crude',
    'XEL': 'Xcel Energy',
    'CVX': 'Chevron',
    'BAC': 'Bank of America',
    'BAH': 'Booze Allen Hamilton'
}

spark = get_spark("stocks")
#grab yfinance data
ticker_lst = ['PL=F', 'GC=F', 'SI=F', 'HG=F', 'PA=F', 'CL=F', 'BZ=F', 'XEL', 'CVX', 'BAC', 'BAH']
dt = yf.download(ticker_lst, start='2020-01-01', group_by='ticker')
#Download historical data for the last year
dt = pd.DataFrame(data=dt)
dt_f = dt.reset_index()
dt_f.columns = ['_'.join(col).strip() for col in dt_f.columns.values] #transform white space to underscore.
dt_f.columns = [''.join(col).replace(' ','_') for col in dt_f.columns.values] #change '=' to underscore.
dt_f = np.round(dt_f, decimals=2)
# print(dt_f.columns)

df_spark = spark.createDataFrame(dt_f)
df_spark = df_spark.withColumn('DateKey', F.date_format(F.col('Date_'),'yyyyMMdd').cast("int"))
df_spark = df_spark.withColumn('Date_', F.date_format(F.col('Date_'),'yyyy-MM-dd'))
df_spark.printSchema()
df_spark.show()
#load into delta table
df_spark.write.mode("overwrite").format("delta").option("inferSchema", "true").saveAsTable("stocks_2")
```

Let’s look at the Spark UI and you can see the Docker Kubernetes tasks that Map Reduce created from the script above. In this case Spark created 8 different tasks to handle this one transformation, this is our Map Reduce. Notice how it broke the data into evenly distributed blocks of data for each task ~66kb.

When you run a Databricks notebook it is creating Docker Images to pull the data hence, the term Distributed Data System.

stocks - Details for Stage 1 (Att: x) Ask Gemini

http://localhost:4040/stages/stage?id=1&attempt=0

FPL Onboarding FPL-HR-Handbook fpl-information-sec... FPL-HR Dr. Keller Portal Muv-WPB MUV | Latest Deals... All Bookm

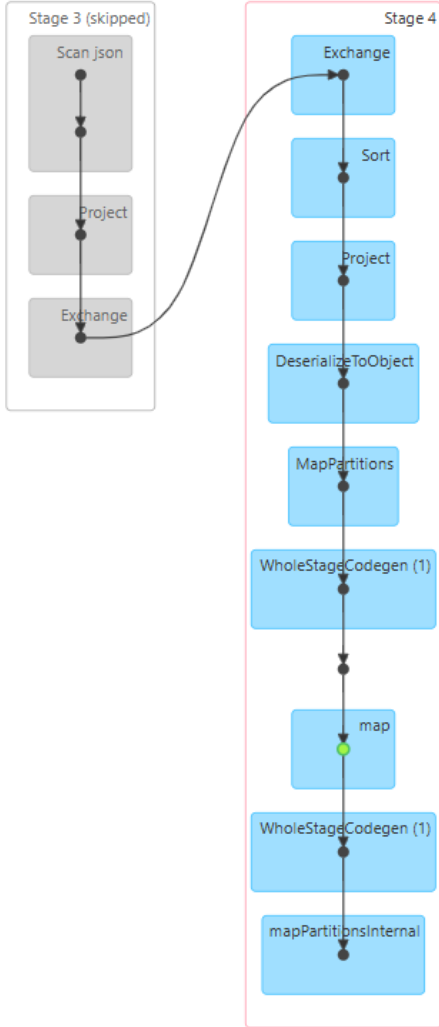
Index	Task ID	Attempt	Status	Locality level	Executor ID	Host	Logs	Launch Time	Duration	GC Time	Shuffle Read Fetch Wait Time	Shuffle Remote Reads	Output Size / Records	Shuffle Write Time	Errors
0	1	0	SUCCESS	PROCESS_LOCAL	driver	kubernetes.docker.internal		2026-08-02 11:06:34	22 s	0.1 s	0.0 ms	0.0 B	68 KIB / 207		
1	2	0	SUCCESS	PROCESS_LOCAL	driver	kubernetes.docker.internal		2026-08-02 11:06:34	22 s	0.1 s	0.0 ms	0.0 B	67.6 KIB / 207		
2	3	0	SUCCESS	PROCESS_LOCAL	driver	kubernetes.docker.internal		2026-08-02 11:06:34	22 s	0.1 s	0.0 ms	0.0 B	67.1 KIB / 207		
3	4	0	SUCCESS	PROCESS_LOCAL	driver	kubernetes.docker.internal		2026-08-02 11:06:34	22 s	0.1 s	0.0 ms	0.0 B	67.3 KIB / 207		
4	5	0	SUCCESS	PROCESS_LOCAL	driver	kubernetes.docker.internal		2026-08-02 11:06:34	22 s	0.1 s	0.0 ms	0.0 B	64.9 KIB / 207		
5	6	0	SUCCESS	PROCESS_LOCAL	driver	kubernetes.docker.internal		2026-08-02 11:06:34	22 s	0.1 s	0.0 ms	0.0 B	66.5 KIB / 207		
6	7	0	SUCCESS	PROCESS_LOCAL	driver	kubernetes.docker.internal		2026-08-02 11:06:34	22 s	0.1 s	0.0 ms	0.0 B	66.5 KIB / 207		
7	8	0	SUCCESS	PROCESS_LOCAL	driver	kubernetes.docker.internal		2026-08-02 11:06:34	22 s	0.1 s	0.0 ms	0.0 B	68.8 KIB / 207		

Showing 1 to 8 of 8 entries

Previous 1 Next

Here is the DAG it created which is made of RDD's Resilient Distributed Datasets. I clicked the "Map" selection in the image to left. It then shows the RDD's that were created for the yfinance pipeline while mapping.

▼ DAG Visualization



▼ DAG Visualization

